

USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

# SparSamp: Efficient Provably Secure Steganography Based on Sparse Sampling

Yaofei Wang, *Hefei University of Technology;* Gang Pei, *Hefei University Of Technology;*
Kejiang Chen and Jinyang Ding, *University of Science and Technology of China;*
Chao Pan, Weilong Pang, and Donghui Hu, *Hefei University of Technology;*
Weiming Zhang, *University of Science and Technology of China*

https://www.usenix.org/conference/usenixsecurity25/presentation/wang-yaofei

This paper is included in the Proceedings of the
34th USENIX Security Symposium.

August 13–15, 2025 • Seattle, WA, USA

978-1-939133-52-6

Open access to the Proceedings of the
34th USENIX Security Symposium is sponsored by USENIX.

ARTIFACT EVALUATED
usenix ASSOCIATION
AVAILABLE

# SparSamp: Efficient Provably Secure Steganography Based on Sparse Sampling

Yaofei Wang* Gang Pei* Kejiang Chen† Jinyang Ding†
Chao Pan* Weilong Pang* Donghui Hu* Weiming Zhang†
*Hefei University of Technology †University of Science and Technology of China

## Abstract

Steganography embeds confidential data within seemingly innocuous communications. Provable security in steganography, a long-sought goal, has become feasible with deep generative models. However, existing methods face a critical trade-off between security and efficiency. This paper introduces SparSamp, an efficient provably secure steganography method based on sparse sampling. SparSamp embeds messages by combining them with pseudo-random numbers to obtain message-derived random numbers for sampling. It enhances extraction accuracy and embedding capacity by increasing the sampling intervals and making the sampling process sparse. SparSamp preserves the original probability distribution of the generative model, thus ensuring security. It introduces only $O(1)$ additional complexity per sampling step, enabling the fastest embedding speed without compromising generation speed. SparSamp is designed to be plug-and-play; message embedding can be achieved by simply replacing the sampling component of an existing generative model with SparSamp. We implemented SparSamp in text, image, and audio generation models. It can achieve embedding speeds of up to 755 bits/second with GPT-2, 5046 bits/second with DDPM, and 9,223 bits/second with WaveRNN.

## 1 Introduction

The vulnerability of digital communication to eavesdropping has been a persistent challenge since the advent of electronic communications. While cryptography effectively conceals message content, it fails to hide the act of transmission itself, and disclosing this metadata can have lethal consequences [12]. Besides, as governments' suspicion of encrypted communication continues to grow around the world (e.g., Kids Online Safety Act in the USA [13], Online Safety Act in the UK [38] etc.), it is essential to adopt more covert communication methods that can complement existing secure communication technologies.

Donghui Hu is the corresponding author.

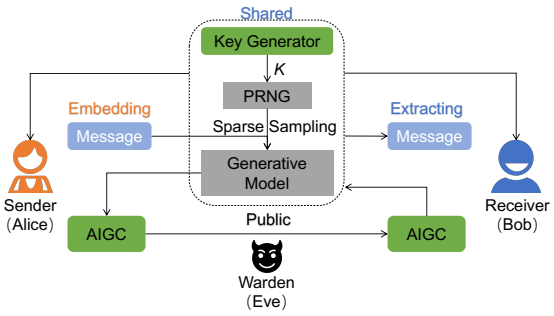



Figure 1: A graphical depiction of steganography.

Steganography [45], the art and science of hiding confidential messages within seemingly innocuous objects. It enables a sender to embed secret messages within ordinary content, ensuring that the communication appears normal, as shown in Figure 1. This technique can help evade even the most oppressive adversaries, as using encryption may raise suspicion. Additionally, steganography facilitates digital "dead-drop" deployments [5], where encoded messages are left on the public internet, allowing the intended recipient to retrieve them without leaving evidence of direct communication.

Traditional steganography typically embeds secret messages directly into existing content (such as images) in a modification manner. One of the most frequently utilized methods in traditional steganography is Least Significant Bit embedding. However, it can be statistically detected with ease [30]. To reduce the number of modifications and fortify the security of steganography, researchers have introduced the matrix-based steganography embedding technique [18, 57, 58]. By considering the impact of modifications at different locations on anti-detection performance, researchers have developed a minimal distortion steganography framework, which has become a seminal framework in the realm of image steganography [17]. However, it is essential to note that the security of these methods cannot be proven and remain susceptible to detection by advanced steganalysis techniques [6].

Although the concept of provably secure steganography (PSS) has long been theoretically feasible [8, 25], achieving this level of security in real-life human communications was considered out of reach due to stringent requirements such as the necessity of a perfect sampler [8, 24, 26, 32, 33]. However, recent advancements in deep learning have led to powerful generative models [20, 21, 31] that serve as samplers and are capable of producing a wide array of artificial intelligence-generated content (AIGC). Notable examples like ChatGPT and DALL-E [43] have demonstrated impressive capabilities in generating high-quality text and images. As AIGC gains popularity across various industries, researchers have begun exploring its potential for steganography, aiming to conceal confidential information within generated content while evading censorship measures [10, 14, 16, 28, 53, 59]. In this context, steganographic schemes aim to make the stego content indistinguishable from regularly generated AI output, rather than from natural training data. This shift in focus makes generative models a valuable tool for implementing PSS.

Yang et al. [53] first introduced the PixelCNN for secure image steganography. Chen et al. [9, 10] utilized text-to-speech generative models for secure audio steganography. Ziegler et al. [59] also proposed a linguistic steganography based on language models. These methods used arithmetic coding (AC) for invertible transformations to embed and extract messages. Kaptchuk et al. [28] highlight the "randomness reuse" issue when directly employing AC, which may lead to potential risk. Then, they proposed a PSS method called Meteor [28], based on the ranged randomness recoverable sampling scheme (RRRSS), a distinct form of AC. However, all of the above methods slightly alter the original probability distributions. Then, Ding et al. [16] proposed a PSS method based on "distribution copies" called Discop. Witt et al. [14] introduced an iterative minimum entropy coupling method (iMEC) to achieve PSS. While these methods successfully maintain the original distribution, their embedding capacity is significantly lower than AC-based steganography's, particularly in low entropy situations. In addition, Discop [16], iMEC [14], and Meteor [28] introduce high computational complexity, with embedding time consumption that is even ten or hundreds of times longer than the inference time of the model, which is unacceptable. In light of these observations, this paper asks

> *How can we embed more message bits with lower complexity without modifying the probability distribution of model inference?*

To achieve the above goal, We need to design a low-complexity, high-embedding-rate steganographic algorithm that is plug-and-play for the generative model. The trained generative model, like the GPT series [7, 37, 41, 42], two steps are usually repeated (as shown in Figure 2): first, it predicts probability distributions for the next token based on the context, and second, it samples from these distributions to generate output. Throughout the generation process, the time spent on sampling is negligible compared to the time spent predicting probability distributions. A key component in implementing PSS with generative models is steganographic coding. Current PSS methods usually replace the sampling process of generative models with steganographic coding to embed messages. For the PSS, the primary goal is to ensure steganography security while maintaining the model's inference capability. Therefore, it is crucial not to alter the probability distribution. Second, according to information theory, the maximum expected number of bits that can be embedded in a token equals the sum of the information entropy of the probabilities. Therefore, steganographic coding should effectively leverage these probability distributions, allowing tokens with lower probability values to carry longer messages. Thirdly, the embedding process should resemble the original sampling to avoid impacting the generation speed. Finally, different messages should result in distinct outputs after embedding to guarantee accurate extraction. In this paper, we combine messages with pseudo-random numbers for sampling to achieve the above requirements.

We know that when we sample once using a random number from the probability distribution inferred by the model, we generate a single token. By conducting multiple samplings with different random numbers, we can produce multiple tokens. When the difference between two random numbers is significant, the likelihood of sampling duplicate tokens decreases. Leveraging this characteristic, we can combine different equal-length messages with a pseudo-random number to get different message-derived random numbers (denoted as **MRN**s) for sampling. In this way, we can preserve the statistical properties of random sampling without altering the model's probability distribution.

We define the sampling interval as the difference between adjacent **MRN**s. The number of **MRN**s, or the length of each embedded message, determines this sampling interval. To avoid different **MRN**s sampling the same result, we need to sparsify the sampling, which will increase the difference between adjacent **MRN**s (the sampling interval). This ensures that the result sampled by the **MRN** originating from the message to be embedded (denoted as **eMRN**) will not overlap with the results sampled by other **MRN**s (referred to as **oMRN**s).

In this paper, we determine the sampling interval based on the number of **MRN**s that previously sampled the same result. This way allows us to incrementally increase the sampling interval and decrease the number of **MRN**s that sample the same token until the result can be uniquely sampled by **eMRN**. Therefore, we refer to this steganographic method as SparSamp.

Through theoretical and experimental evidence, we have demonstrated that SparSamp can embed messages of varying lengths without altering the probability distribution predicted by the generative model. The embedding rate achieved by

SparSamp, measured in bits per token, closely approximates the information entropy of the probability distribution. Additionally, the extra computational overhead introduced by SparSamp during each sampling operation is minimal, with a complexity of only $O(1)$, ensuring that the original model's normal generation process remains unaffected, preserving both speed and quality. Moreover, SparSamp adopts a plug-and-play design; users can achieve message embedding by simply replacing the sampling component in the generative models. In this paper, SparSamp has been successfully implemented in various models, including large language models (LLMs) such as GPT-2 [42], Qwen-2.5 [51], Llama-3 [1]. It has also been used in image generation models like the Denoising Diffusion Probabilistic Model (DDPM) [11] and text-to-speech models like WaveRNN [27].

The main contributions of this paper can be summarized as follows:

- **A novel approach to embedding messages.** We propose an innovative method for embedding messages by transforming them into message-driven sampling. This approach combines messages with pseudo-random numbers for sampling, eliminating the need to adjust probability distributions or sorting. As a result, our method ensures steganography security while ensuring efficient message embedding.
- **Strategy for accurate decoding and improving embedding rate.** We employed a technique called "Sparse Sampling" to achieve accurate decoding and a high embedding rate. This technique involves determining the number and interval of current samples based on how many **MRN**s produced the same result in the previous sampling step. This technique helps ensure that the sampling results are not mistakenly attributed to multiple **MRN**s while adaptively distributing the message across consecutive sampling results.
- **Achieving $O(1)$ time complexity.** The longer the message length for each embedding in SparSamp, the denser the sampling becomes, resulting in more **MRN**s sampling the same outcome. This could potentially lead to increased time complexity. In this paper, we address this issue by achieving an $O(1)$ time complexity through the homogenization of sampling. Our approach requires only two mathematical operations to determine the number of **MRN**s sampled the same outcome. This effectively reduces the impact on the generation speed of the model when using SparSamp.
- **Capability of detecting token ambiguity.** Although the tokens LLMs are not uniquely decodable, SparSamp can detect token ambiguity before extracting a segment of the message, aiding in correct parsing.
- **Benchmarking and comparison.** SparSamp is easy to deploy. We have deployed it in three generative services, including text, audio, and image generation, and compared it with other methods, demonstrating its excellent performance in speed, capacity, and security.

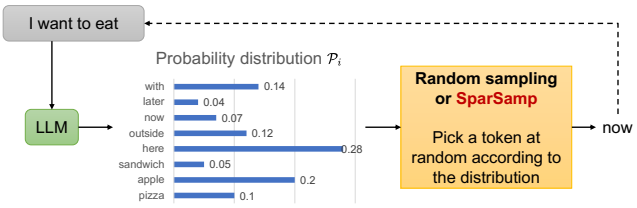



Figure 2: A generation process of LLM

## 2 Background and Related Work

### 2.1 Deep Generative Models

Achieving PSS requires creating a perfect simulation of the cover distribution. However, due to the complexity of digital media such as traditional photographs and human-written text, this level of perfection has been historically unattainable. Nevertheless, recent advances in generative models for natural languages [7, 41, 42], images [15, 44], and sounds [4, 46] present promising opportunities for achieving perfect security in real-world applications. These models utilize well-defined sampling mechanisms to generate content that closely mimics human-created output.

This paper will primarily use LLMs to illustrate our proposed method. In recent years, significant advancements have been made in the LLM, primarily due to the introduction of the Transformer model [48] and the success of the GPT series models [7, 37, 41, 42]. LLMs have demonstrated remarkable capabilities in chatbot development, machine translation, and code generation tasks.

The LLM employs a vocabulary $\mathcal{V}$ consisting of tokens (which can be words or word fragments), typically containing around 50,000 tokens or more [42]. For next-token ($v_i$) prediction, an LLM is represented as a neural network function $\mathcal{G}$ that takes a sequence of known tokens $v_{<i}$ as input and produces a probability distribution $\mathcal{P}_i = p(v_i \mid v_{<i}) = \mathcal{G}(v_{<i})$ over the vocabulary using a softmax function. Subsequently, random sampling is often utilized to select a token based on this probability distribution, as illustrated in Figure 2.

### 2.2 Information-theoretic Security of Steganography

Information-theoretic security is highly valued in steganography, as it offers robust protection against various forms of detection [8]. Assuming that both communicating parties have access to the exact distribution of the object $x$, security is defined by the Kullback-Leibler divergence (KLD) between

Table 1: Comparison of different methods

| Method | Computational complexity | Probability unchanged |
|---|---|---|
| AC-based [10,47,53,59] | $O(N)$ | × |
| Meteor [28] | $O(N), O(2^{\lceil H(\mathcal{P})\rceil}N)$ | × |
| ADG [56] | $O(N\log N)$ | × |
| Discop [16] | $O(1), O(N)$ | ✓ |
| iMEC [14] | $O(N\log N)$ | ✓ |
| **SparSamp** (proposed) | $O(1)$ | ✓ |

the cover distribution $P_c$ and the stego distribution $P_s$. This divergence can be calculated using the following formula:

$$D_{\mathrm{KL}}(P_{\mathrm{c}}\|P_{\mathrm{s}}) = \sum_{\mathbf{x}\in\mathcal{C}} P_{\mathrm{c}}(\mathbf{x})\log\frac{P_{\mathrm{c}}(\mathbf{x})}{P_{\mathrm{s}}(\mathbf{x})}. \tag{1}$$

The KLD essentially measures how different two distributions are - the smaller the value, the more similar they are. When it equals zero, the distributions are identical, making it impossible to distinguish between cover and stego. Under these conditions, a steganalyzer would have no advantage over random guessing. Therefore, in PSS based on generative models, it is essential to ensure both the invariance of the model's predictive probabilities and the randomness of the sampling process.

## 2.3 Attempts to Provably Secure Steganography in the AIGC Era

Several methods have been proposed for realizing PSS by utilizing the probability distribution provided by generative models. In the following, we will analyze the characteristics of each method concerning computational complexity, security, and embedding rate. The comparison of different methods is shown in Table 1.

**AC-based steganography:** Arithmetic coding (AC) is a data compression technique designed to encode sequences of elements with known probability distributions. This method is particularly effective for long sequences, as it can achieve compression close to the information entropy value. Ross Anderson [3] suggested that a steganography scheme can achieve perfect security if there is a perfect compression scheme. Following this, Le et al. [47] developed coding schemes called P-codes using AC. At that time, however, practical samplers were not available. It was not until the advent of deep learning that researchers began to explore new AC-based steganographic schemes. Yang et al. [53] introduced an autoregressive generative model, PixelCNN to create a secure image steganography scheme. Chen et al. [10] extended this method from image generation to text-to-speech applications. Additionally, Ziegler et al. [59] proposed a steganography method based on AC that utilized LLMs.

AC-based steganography expresses messages through the shared prefix of the sampled result interval. This approach provides high embedding rates, approaching the information entropy for long messages. However, it necessitates updating the probability intervals after each sampling step, which leads to a time complexity of $O(N)$, where $N$ is the number of candidate tokens. Moreover, precision limitations during these updates can modify the original distribution, which may compromise security.

**Meteor:** Kaptchuk et al. [28] pointed out an issue with the AC-based scheme known as the *reuse of randomness* problem. When a message is not re-encrypted each time it is used for sampling, there is a risk of the stego being detected, potentially leading to information leaks. To address this issue, they introduced a new steganography scheme called Meteor. However, we disagree with this perspective because the AC-based steganography scheme updates the probability of the next token to avoid the reuse of randomness (this falls outside the scope of this paper).

Meteor embeds messages like AC-based steganography but doesn't update probability intervals, and the message is re-encrypted before each sampling, thereby avoiding the reuse of randomness. However, treating each sample independently leads to shorter or no shared prefixes, resulting in lower embedding rates. To improve this, Meteor reorders probabilities, which introduces $O(2^{\lceil H(\mathcal{P})\rceil}N)$ complexity, where $H(\mathcal{P})$ is the entropy in the distribution. Additionally, due to limitations in calculation precision, Meteor truncates probabilities, which may impact security.

**ADG:** Zhang et al. [56] proposed a steganography method that utilizes adaptive dynamic grouping (ADG). ADG dynamically groups the probability distribution of all tokens in the vocabulary at each time step into $2^r$ groups, with each group having approximately the same total probability. Each group represents message bits of length $r$, and all tokens within a group are associated with the same message bits. To embed a message, a random sample is taken from the normalized distribution of the selected group to generate the next token.

ADG assumes that the message bits follow a uniform distribution, meaning each group has an equal probability of being selected. In theory, perfect security can be achieved only if the grouping is perfectly balanced. However, given the discrete nature of probability distributions, achieving this balanced grouping is highly unlikely. Consequently, the actual distribution used for embedding messages is a balanced distribution, which often deviates from the original distribution. Additionally, the runtime for dynamic grouping is $O(N\log N)$, which adds significant computational complexity to the process.

**Discop:** Ding et al. [16] proposed a steganography scheme based on "distribution copies" named Discop. In this scheme, several "distribution copies" are generated by rotating all intervals by specific displacements. At each time step, the message determines which "distribution copy" to sample from. To enhance the embedding rate, Discop decomposes the multi-

variate distribution into multiple bivariate distributions using a Huffman tree, constructing "distribution copies" for each bivariate distribution recursively.

Discop strictly maintains the original distribution and thus achieve PSS due to the probability of each token in different "distribution copies" is equal. By constructing Huffman trees, Discop can achieve high embedding rates. However, the complexity of creating a Huffman tree is $O(N)$, and the average complexity of walking from the root node to a leaf node of the tree is $O(\log N)$. Therefore, Discop incurs certain time overhead.

**iMEC:** Witt et al. [14] analyzed information-theoretic steganography using the concept of minimum entropy coupling. They proved that perfect steganographic security is equivalent to a coupling problem and that achieving maximum transmission efficiency in a perfectly secure system is equivalent to a minimum entropy coupling problem. Their proposed iMEC scheme fully leverages the properties of coupling and minimum entropy coupling.

Similar to Discop, iMEC theoretically does not disrupt the probability distribution and thus achieve PSS. However, iMEC does have a certain bit error rate. To achieve minimum entropy coupling and enhance the embedding rate, a considerable amount of computational complexity, specifically $O(N \log N)$, is necessary to couple the probabilities.

Based on the previous analysis, we can find that the increase in computational complexity stems from operations such as scaling [59], reordering [16, 28, 56], and coupling the probability distribution [14]. The destruction of the probability distribution is attributed to two factors: (1) truncation of probabilities [56] and (2) limitations in computational precision [28, 59]. In this paper, we aim to design an efficient message embedding scheme that can embed more message bits with lower complexity without changing the probability distribution of model inference.

## 3 SparSamp Methodology

In this section, we will first introduce a novel message embedding method based on message-driven sampling. Our approach embeds message segments sequentially, enabling efficient handling of arbitrary-length messages. We then demonstrate how to achieve $O(1)$ computational complexity per sampling step, ensuring no degradation in model's generation speed. Finally, we will address the issue of token ambiguity in LLMs.

### 3.1 Message Embedding via Message-Driven Sampling

To achieve message-driven sampling without destroying the distribution, we borrowed the idea of using pseudo-random numbers from Discop [16] to perform random sampling. Let $r_i \sim U[0,1)$ denote the initial pseudo-random number generated by a pseudo-random number generator (PRNG) at step $i$. We define:

- $\mathcal{V}$: the candidate vocabulary
- $v_i \in \mathcal{V}$: the sampled token at step $i$
- $\mathcal{P}_i$: the probability distribution at step $i$, sorted according to the model's default order

The sampling of token $v_i$ based on $\mathcal{P}_i$ using $r_i$ is represented by the following sampling function:

$$\mathcal{S}(\mathcal{P}_i, r_i) = v_i, \tag{2}$$

And the corresponding algorithm is shown in Algorithm 1.

**Algorithm 1:** $\mathsf{sample}(\mathcal{P}_i, r_i)$: Sampling in the probability distribution of candidate tokens using a random number

**Input:** The pseudo-random number $r_i$, Probability distribution $\mathcal{P}_i$

$cuml \leftarrow 0$, $SE_i \leftarrow [0,0]$
**for** $k \leftarrow 0$ **to** $|\mathcal{P}_i| - 1$ **do**
  $cuml \leftarrow cuml + \mathcal{P}_i(k)$
  **if** $cuml > r_i$ **then**
    $v_i \leftarrow$ corresponding to the $k$-th token (in default order);
    $SE_i \leftarrow [cuml - \mathcal{P}_i(k), cuml]$;
    **break**;
  **end**
**end**

**Output:** Sampled token $v_i$, Start and end positions of the sampled token $SE_i$

To combine the message with the pseudo-random number, we transform a message $m$ with the length of $l_m$ into a number in the interval $[0,1)$ using a function $\mathsf{bin2num}()$, defined as:

$$\mathsf{bin2num}(m) = \frac{\mathsf{bin2dec}(m)}{2^{l_m}}, \tag{3}$$

where $\mathsf{bin2dec}(m)$ is $m$'s decimal representation. We then use modular addition to combine $\mathsf{bin2num}(m)$ with the pseudo-random number $r_i$, obtaining the "message-derived random number (**MRN**)", denoted as $r_i(m)$:

$$r_i(m) = [r_i + \mathsf{bin2num}(m)] \bmod 1. \tag{4}$$

We distinguish between two types of **MRN**:

- **eMRN**: The **MRN** of the actual message $m$ to be embedded
- **oMRN**: **MRN** of other messages $m'$ that have the same length as the embedded message $m$

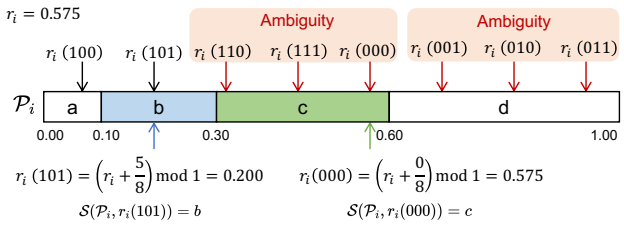



Figure 3: An example of sampling based on **MRN**s. The initial pseudo-random number is $r_i = 0.575$. The message length $l_m$ to be embedded is 3, and the **MRN**s are $r_i(xxx)$.

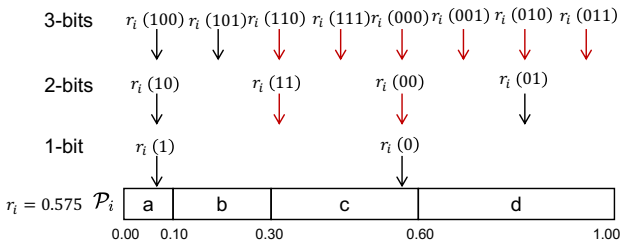



Figure 4: Sampling based on different number/length ($l_m$) of **MRN**s/message. The red arrow represents sampling that causes decoding ambiguity.

We transform message embedding into message-driven sampling by replacing pseudo-random numbers with **eMRN**s for sampling. An example of this process is illustrated in Figure 3. Let's assume the pseudo-random number is $r_i = 0.575$ and the message length to be embedded is $l_m = 3$. We can generate eight ($2^{l_m} = 8$) **MRN**s based on the length of the message. For embedding the messages "100" or "101", tokens "a" or "b" can be sampled according to **eMRN** $r_i(100)$ or $r_i(101)$, respectively. The receiver, who knows both the message bit length and pseudo-random numbers, can decode "100" or "101" based on the sampled tokens "a" or "b". This decoding is achievable because **eMRN** uniquely sampled these specific tokens.

**Decoding ambiguity** However, the above embedding has the problem of causing "ambiguity" in the decoding. We define "ambiguity" as the receiver's inability to determine which specific **MRN** led to the observed sampling result. This ambiguity arises because multiple **MRN**s can potentially sample the same token. To illustrate this point, consider the sampling indicated by the orange block in Figure 3. When embedding the message "000", sampling based on the **eMRN** $r_i(000)$ produces "c". However, sampling based on **oMRN**s $r_i(110)$ and $r_i(111)$ also results in "c". As a result, the receiver has no way to know which **MRN** led to "c", impeding the extraction of the embedded message.

**Conflict** Our analysis reveals that successful message embedding and extraction hinges on the uniqueness of the sampling result caused by **eMRN**. To mitigate ambiguity, one could reduce the message embedding length, but this approach compromises the embedding rate. Figure 4 illustrates this trade-off: 1) Shorter messages (e.g., 1-bit) create larger sampling interval between adjacent **MRN**s, reducing ambiguity rate. 2) Longer messages increase the probability of multiple **MRN**s sampling the same token (indicated by red arrows). This scenario underscores an inherent **conflict** between message length and extraction accuracy.

To address this decoding ambiguity and enhance the embedding length, we borrow the idea of scaling from the iMEC [14] and the AC-based method [47, 59]. But instead of coupling or scaling probabilities, which can increase complexity and distort distributions, we scale the sampling interval by increasing the distance between neighbouring **MRN**s. When the interval between adjacent **MRN**s is sufficiently large, it becomes highly improbable to sample the same token, effectively avoiding ambiguity. Therefore, ambiguity can be avoided. Next, we will introduce which **MRN**s to choose to increase the sampling interval.

## 3.2 Sparse Sampling

Our previous analysis demonstrated that multiple **MRN**s may sample the same token, as illustrated in Figures 3 and 4. In such cases, the receiver can only infer a range of possible messages. To narrow this range, we record all **MRN**s that sample the same token as the **eMRN** and use these recorded **MRN**s for subsequent sampling steps. Consequently, we need to update the transformation function $\mathsf{bin2num}()$ (Equation (3)) for converting message into number as follows:

$$\mathsf{bin2num}(m) = k_m / N_m \tag{5}$$

where $N_m$ is the number of candidate messages, and $k_m$ denotes the index of the message to be embedded in the candidate messages, and $k_m \in [0, N_m)$.

**Message Embedding as Index Embedding** Our embedding process can be conceptualized as embedding an index within a set of candidate messages. According to Equations (3) and (5), the embedded message represents its index among all possible candidates, which the receiver is also aware of.

Figure 5 illustrates this concept: when embedding the message "111" in step $i$ with the pseudo-random $r_i = 0.575$, we have $k_m = 7$, $N_m = 8$. The corresponding **eMRN** is $r_i(111) = 0.450$. Based on the sampled token "c", the receiver can determine that the embedded message is one of "110", "111" or "000" at this step. To clarify to the receiver which message we have embedded, we use these three candidate messages for sampling in the next step. Consequently, in the next step $i+1$, the index of the message to be embedded within the set of candidate messages is updated to $k_m = 1$ and

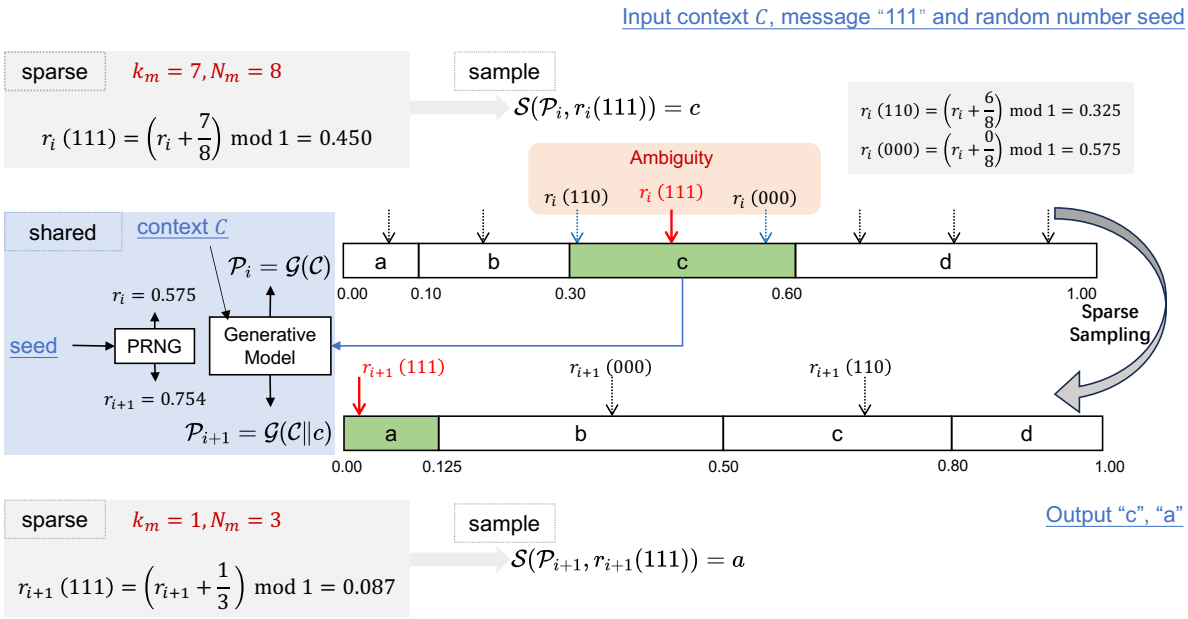



Figure 5: An example of embedding based on Sparse Sampling.

$N_m = 3$. To avoid the repeated use of randomness, we use a new pseudo-random number $r_i$ at each step. Using Equation (4), we can derive the new **eMRN**, $r_{i+1}(111) = 0.087$, for sampling in step $i+1$. At this point, only the token "a" will be sampled based on this **eMRN**, allowing the message "111" to be uniquely decoded.

Our objective is to gradually narrow down the range of candidate messages until the receiver can clearly identify the embedded message (when $N_m = 1$). Initially, when $N_m$ is large, a fraction of **oMRN**s will sample the same token as **eMRN**, which leads to a reduction in $N_m$ in the subsequent round. This process effectively reduces the range of possible embedded messages. As $N_m$ decreases, the distance between adjacent **MRN**s, given by $\frac{1}{N_m}$, increases. This results in sparser sampling, thereby reducing decoding ambiguity. Due to this characteristic, we name this approach **SparSamp**.

## 3.3 Achieving $O(1)$ time complexity

The time required by SparSamp is mainly attributed to the updates of $N_m$ and $k_m$ to get **eMRN**. When we update $N_m$ and $k_m$ by comparing and counting, longer embedded messages lead to an increased number of candidate messages that need to be compared and counted. This results in higher time complexity, which is not desirable. Therefore, based on the characteristic of uniform sampling, $N_m$ and $k_m$ can be calculated using only two computations.

As $k_m$ represents the position of **eMRN** within **MRN**s that sampled to the same token. Since the difference between adjacent **MRN**s is $\frac{1}{N_m}$, as indicated in Equations (3)-(5), we can obtain $k_m$ and $N_m$ by calculating the distance from **eMRN** to the starting point $SE_i(0)$ and the end point $SE_i(1)$ of the sampled token. The relevant calculations are presented in Algorithm 2, which we refer to as **Sparse**. As a result, SparSamp introduces only $O(1)$ additional time complexity in each sampling step.

**Algorithm 2:** sparse$(SE_i, N_m, k_m, r_i)$: Sparse the number and interval of sampling by updating the parameters of the number of candidate messages and the index of the message to be embedded

**Input:** Start and end positions of sampled token $SE_i$, The number of candidate messages $N_m$, The index of the message to be embedded $k_m$, The pseudo-random number $r_i$

$temp_0 = \lceil (SE_i(0) - r_i) * N_m \rceil$
$temp_1 = \lceil (SE_i(1) - r_i) * N_m \rceil$
**if** $k_m + r_i * N_m >= N_m$ **then**
  $k_m = k_m - N_m - temp_0$
**else**
  $k_m = k_m - temp0$
**end**
$N_m = temp_1 - temp_0$

**Output:** The updated $N_m$ and $k_m$

## 3.4 Overview of SparSamp

Figure 6 illustrates the overview of SparSamp. The sender and receiver must share the same settings, which include the initial context, the PRNG seed, and the generative model. This

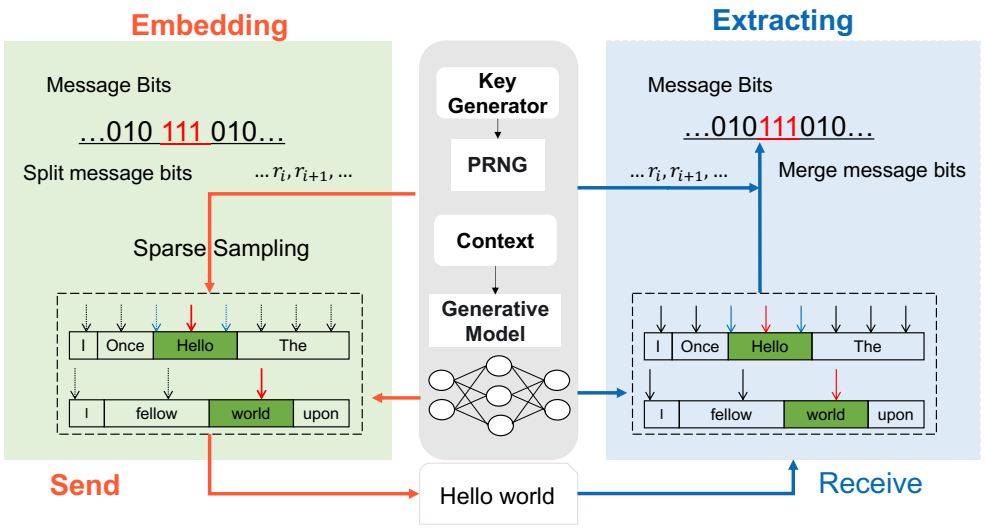



Figure 6: Overview of SparSamp. To ensure effective covert communication, both the sender and receiver need to have the same generative model, context, pseudo-random number generator (PRNG), and key. The sender continuously samples the next token with the message and pseudo-random number, until the entire message is embedded. Once the stego is created, it is sent to the receiver. The receiver can then synchronize with the sender's states and extract the message by using the reverse process.

synchronization ensures that both parties maintain consistent states throughout the process. It is important to note that it is unnecessary to transmit the model itself because many large models are readily available to the public on platforms like Hugging Face and GitHub. Therefore, both parties only need to agree on which model to use and then load it separately.

Algorithm 3 demonstrates how SparSamp embeds messages during the model generation process. The whole message is divided into equal chunks of size ($l_m$) and embedded sequentially. When the model samples the EOS (end-of-sequence) token, any unembedded messages along with their associated states $(N_m, k_m)$ are carried over to the next conversation round. Conversely, if the entire message is embedded before the model terminates, the normal sampling continues until termination occurs naturally. It's important to note that SparSamp does not alter the model's generation process; it only replaces the original sampling method without affecting the model's termination conditions. With SparSamp, once the message and random number are determined, the corresponding output of the model based on each message is fixed and unique. The corresponding extracting process is shown in Algorithm 4.

**Advantages** In summary, SparSamp offers the following advantages.:

1. Preservation of Distribution: It maintains the original probability distribution $\mathcal{P}_i$, ensuring statistical consistency.

2. Sampling Randomness: The method retains the inherent randomness of the sampling process, which is crucial for security.

3. Computational Efficiency: By eliminating the need for sorting or combining probability distributions, it enables rapid embedding and decoding.

4. Resumable embed and extract: Although SparSamp embeds and extracts messages in segments, if a segment of the message is not fully embedded or extracted, it can be resumed in the next stage. The variables in parentheses shown in Algorithm 3 and 4, such as $N_m$ and $k_m$, serve as input and output parameters that facilitate this resumable functionality.

## 3.5 Dealing with Token Ambiguity

For state-of-the-art PSS methods [14, 16, 28, 56, 59], including SparSamp, successful decoding relies on the token path generated by the sender and receiver matching precisely at the encoding and decoding process. However, for LLMs such as GPT-2, the vocabulary is not prefix-free due to the use of byte-pair encoding (BPE) [19]. Consequently, token ambiguity (TA) may occur. This ambiguity can be mitigated by utilizing alternative tokenizers, such as word-based or character-based tokenizers, or by employing disambiguating techniques [5, 36, 40, 50]. Given that token disambiguation techniques are plug-and-play and generalizable, better steganographic coding in combination with these token disambiguation techniques will also lead to better results.

Currently, only two disambiguating methods can be utilized without compromising the probability distribution, namely,

**Algorithm 3:** The main loop of SparSamp's message embedding algorithm

**Input:** Context $\mathcal{C}$, Generative Model $\mathcal{G}$, PRNG, message to embed $M$, Length of the message to be embedded each time $l_m$, $(N_m, k_m)$

$v \leftarrow$ " ", $\mathcal{S} \leftarrow$ " ", $count \leftarrow 0$, $(N_m \leftarrow 1)$
**while** *v is not "EOS"* **do**
  // Terminate the embedding process until the model samples the end-of-sequence token (EOS)
  $r \leftarrow$ PRNG.next()
  $\mathcal{P} \leftarrow \mathcal{G}(\mathcal{C})$ // Model infers the probability distribution based on the context
  **if** $count \times l_m < |M|$ **then**
    **if** $N_m = 1$ **then**
      $k_m \leftarrow$ bin2dec$(M[count \times l_m : (count+1) \times l_m - 1])$ // Convert a portion of the $l_m$-bit binary message in $M$ to the decimal $k_m$
      $N_m \leftarrow 2^{l_m}$
      $count \leftarrow count + 1$
    **end**
    $r_m \leftarrow \left(\frac{k_m}{N_m} + r\right) \bmod 1$;
    $v, SE \leftarrow$ sample$(\mathcal{P}, r_m)$
    $N_m, k_m \leftarrow$ sparse$(SE, N_m, k_m, r)$
  **else**
    $v \leftarrow$ sample$(\mathcal{P}, r)$ // Normal sampling
  **end**
  $\mathcal{S} \leftarrow \mathcal{S} \| v$ // Update stego
  $\mathcal{C} \leftarrow \mathcal{C} \| v$ // Update context
**end**

**Output:** Stego $\mathcal{S}$,$(N_m, k_m)$

**Algorithm 4:** The main loop of SparSamp's message extracting algorithm

**Input:** Context $\mathcal{C}$, Stego $\mathcal{S}$, Generative Model $\mathcal{G}$, PRNG, Length of the message to be embedded each time $l_m$, $(N_m, temp_0^{arr}, N_m^{arr})$

$M \leftarrow$ " ", $(N_m \leftarrow 2^{l_m}, temp_0^{arr}, N_m^{arr} \leftarrow [])$
**for** *v in* $\mathcal{S}$ **do**
  $r \leftarrow$ PRNG.next()
  $\mathcal{P} \leftarrow \mathcal{G}(\mathcal{C})$ // Model infers the probability distribution based on the context
  Get the $SE$ of $v$ based on $\mathcal{P}$
  $\mathcal{C} \leftarrow \mathcal{C} \| v$ // Update context
  $temp_0 = \lceil (SE(0) - r) * N_m \rceil$
  $temp_1 = \lceil (SE(1) - r) * N_m \rceil$
  $temp_0^{arr}.append(temp_0)$
  $N_m = temp_1 - temp_0$
  $N_m^{arr}.append(N_m)$
  **if** $N_m = 1$ **then**
    $count = |temp_0^{arr}| - 2$
    $k_m \leftarrow temp_0$
    **while** $count \geq 0$ **do**
      $k_m \leftarrow temp_0^{arr}(count) + ((k_m + N_m^{arr}(count)) \bmod N_m^{arr}(count))$
      $count \leftarrow count - 1$
    **end**
    $k_m \leftarrow ((k_m + 2^{l_m}) \bmod 2^{l_m})$
    $M = M \| \text{dec2bin}(k_m, l_m)$ // Convert the decimal $k_m$ into an $l_m$-bit binary message and add it to $M$
    $N_m \leftarrow 2^{l_m}$
    $temp_0^{arr}, N_m^{arr} \leftarrow []$
  **end**
**end**

**Output:** Message $M$, $(N_m, temp_0^{arr}, N_m^{arr})$

backtracking with checkpoints (BackCheck) [5] and SyncPool [40]. Each of these methods has its pros and cons. SyncPool does not affect the complexity of generation and decoding but diminishes the embedding rate. In contrast, BackCheck inserts checkpoints at each segment of the message, which has a lesser impact on the embedding rate and does not alter the generation process; however, it requires repeated backtracking to identify the correct token path during extraction, thus increasing the complexity on the extraction side.

BackCheck was initially combined with the AC-based approach [5]. We believe that SparSamp is better suited to work with BackCheck for several reasons. First, BackCheck does not influence the embedding complexity and embedding rate. Second, a sparser sample results in fewer tokens being selected for the random number determination, which significantly reduces the amount of backtracking needed. Finally, when the token path used in SparSamp decoding is not consistent with the token path adopted in embedding, according to Algorithm 4, there may be a situation where $N_m = 0$, which means that the message cannot be extracted, and this situation will only occur after the TA has appeared, according to which we can infer the approximate location of the TA's appearance ahead of time for backtracking, instead of needing to backtrack after the message has been extracted.

## 3.6 Proof of Security

SparSamp ensures the security of steganography in two important ways, as outlined in Section 3.1. First, it preserves the original probability distribution at each step for sampling. Second, it maintains randomness in the sampling process within this distribution throughout each step.

**Theorem 1** *For any polynomial-time distinguisher $\mathcal{A}$, it is computationally infeasible to distinguish between samples drawn from $\mathcal{P}_i$ using $r_i$ and using $r_i(m)$.*

**Proof 1** *We prove this theorem as follows.*

1. ***Pseudo-random number generator:*** *In this paper, we use pseudo-random numbers for sampling. Let PRNG be a deterministic polynomial-time algorithm such that for any $\lambda$-bit input $s \in \{0,1\}^{\lambda}$, the algorithm PRNG outputs a bit string of length $\ell(\lambda)$, where $\ell$ is a polynomial. For all $\lambda$, it holds that $\ell(\lambda) > \lambda$. For all probabilistic polynomial-time distinguishers $\mathcal{A}$, there exists a negligible function negl such that the following inequality holds:*

$$\left|\Pr[\mathcal{A}(PRNG(s)) = 1] - \Pr\left[\mathcal{A}\left(s'\right) = 1\right]\right| \leq \mathrm{negl}(\lambda) \tag{6}$$

*Where the seed $s$ is uniformly chosen from $\{0,1\}^{\lambda}$, and $s'$ is uniformly chosen from $\{0,1\}^{\ell(\lambda)}$, both being true random bit strings.*

2. ***The precision of the pseudo-random number:*** *In practice, the precision of the pseudo-random numbers we use is consistent with the pseudo-random numbers used in the model, which is actually the same PRNG. Assuming the precision is $h$ bits, at each time step $i$, consecutive $h$ bits $b_{hi}, b_{hi+1}, \ldots, b_{hi+h-1}$ are sequentially taken from $PRNG(s)$, and by calculating a pseudo-random number within the interval $[0,1)$, $r_i = \frac{\sum_{t=0}^{h-1} b_{hi+t} \times 2^t}{2^h}$. Thus, for all $a \in \left\{i \times 2^{-h}\right\}_{i=0,\ldots,2^h-1}$, there exists a negligible function $\varepsilon$ with respect to the security parameter $\lambda$ such that the following inequality holds:*

$$\begin{aligned} \left|\Pr\left[r^{(t)} = a\right] - 2^{-h}\right| &\leq \varepsilon(\lambda) \\ 2^{-h} - \varepsilon(\lambda) \leq \Pr\left[r^{(t)} = a\right] &\leq 2^{-h} + \varepsilon(\lambda) \end{aligned} \tag{7}$$

3. ***Computationally indistinguishable:*** *Since $\frac{k_m}{N_m} \in [0,1)$, the sum $r_i + \frac{k_m}{N_m}$ lies in $[0,2)$. Applying the modulo operation ($\bmod 1$) ensures that $r_i(m)$ remains in $[0,1)$. Importantly, for $a \in \left\{t \times 2^{-h}\right\}_{t=0,\ldots,2^h-1}$, we have*

$$\sum_a \left|\Pr[r_i(m) = a] - \Pr[r_i = a]\right| = 2^{h+1} \cdot \varepsilon(\lambda) \tag{8}$$

*Since $h$ is a constant independent of $\lambda$, there exists a negligible function $\varepsilon'(\lambda) = 2^{h+1} \cdot \varepsilon(\lambda)$ with respect to $\lambda$ such that the following inequality holds:*

$$\sum_{a \in \left\{t \times 2^{-h}\right\}_{t=0,\ldots,2^h-1}} \left|\Pr[r_i(m) = a] - \Pr[r_i = a]\right| \leq \varepsilon'(\lambda) \tag{9}$$

*Given that the only difference between the stego and the cover is the random variable used during the sampling process, and the distributions of $r_i(m)$ and $r_i$ are computationally indistinguishable, it follows that the stego and cover distributions are also computationally indistinguishable.*

## 4 Evaluation

In this section, we will conduct experiments and compare SparSamp with the previous steganography methods pursuing provable security, namely AC-based [59], Meteor [28], ADG [56], Discop [16], and iMEC [14].

### 4.1 Experimental Setup

**Generative Models**: In this study, we employ a diverse range of generative models, including LLMs (GPT-2 [42], Qwen-2.5 [51], Llama-3 [2]), image generation model (DDPM [11]), and audio generation model (WaveRNN [27]). To enhance the generation process, we implement two sampling techniques:

- Top-$k$ sampling [23]: This method selects the next word from the $k$ most probable choices in the vocabulary. It limits the selection to a fixed number of top candidates.
- Nucleus sampling (or top-$p$ sampling) [22]: This is a widely adopted technique in generation tasks that constrains the vocabulary size from which we sample. Instead of using a fixed number like top-$k$, it dynamically selects from the smallest words whose cumulative probability exceeds the threshold $p$.

We utilize a range of truncation parameters to assess performance across various scenarios: $p = \{0.80, 0.95, 1.00\}$, and $k = [2:4:98]$. This approach allows us to evaluate the steganography performance under different levels of vocabulary restriction.

**Generative Tasks**: We deploy the steganographic algorithm across three generative tasks:

- Text generation. We employ three pre-trained LLMs: GPT-2 version [42], Qwen-2.5 [51] and Llama-3 [2]. From the IMDB dataset [34], we randomly select 100 text samples and generate approximately 100 to 200 tokens based on the first three sentences of each sample.
- Image generation. We employ a learning-free method for controlling the generation of unconditional DDPM [11], which is trained on FFHQ dataset [29]. We quantize the continuous probabilities from the last layer into discrete probabilities that correspond to 256 pixel values for sampling, which are then saved as 8-bit images based on the StegaDDPM approach [39]. A total of 100 pairs of images are generated for evaluation.

- Text-to-speech (TTS). Using pre-trained Tacotron [49] and WaveRNN [27] models, we generate approximately 3-second speech segments corresponding to the first sentence of 100 text samples from the IMDB dataset.

**Token Ambiguity (TA)**: For a fair comparison, we first evaluate performance using sentences that do not have TA in subsections 4.3-4.5. We then address TA based on SynPool [40] and BackCheck [5] in subsection 4.8.

All experiments were conducted using consistent hardware configurations: an Intel Xeon Gold 6330 CPU (2.00GHz), 128GB RAM, and an NVIDIA GeForce RTX 4090 GPU. To ensure uniformity, all model queries were executed on a single GPU. We employ the default double-precision floats in Python for our numerical computations.

## 4.2 Metrics

We evaluate the performance of SparSamp using metrics that assess the efficiency of the steganography in terms of time/speed, capacity, and security.

- Time/Speed: To measure the complexity and embedding efficiency of different steganographic algorithms, we use the following metrics:
  - Average Token Sampling Time (ATST): The mean time taken to sample each token during the sampling phase.
  - Sampling-to-Inference Time Ratio (SITR): The ratio of sampling time to model inference time.
  - Generation Speed: The number of tokens generated per second.
  - Embedding Speed: The number of bits embedded per second.
  - Decoding Speed: The number of bits decoded per second.
- Capacity: We use two metrics to describe the capacity of a steganography method:
  - Embedding rate: The average number of bits of information embedded in each generated token.
  - Utilization rate of entropy (Utilization): Following [16], we use this metric to measure embedding ability. It is defined as the ratio of the total embedded message length to the entropy sum over all time steps, indicating how close the embedding rate is to its theoretical limit.
- Security: We use two KLD metrics:
  - Average KLD (Ave KLD): The mean KLD over all time steps is calculated by dividing the cumulative KLD by the total number of tokens. It indicates the average extent to which the steganography method alters the original distribution. Lower values are better.
  - Maximum KLD (Max KLD): The highest KLD value across all time steps, indicating the most severe alteration to the original distribution. Lower values are better.

## 4.3 Evaluating SparSamp with Different $l_m$

SparSamp embeds the whole message by dividing it into submessages with length $l_m$, as illustrated in Algorithm 3. We evaluated varying lengths $l_m$ on GPT-2 using top-$p$ sampling (with $p = 1.00$). The results shown in Table 2 indicate that, due to the limitations of double-precision calculations, the maximum length $l_m$ can reach is 1023. As long as $l_m$ does not exceed 1023, we can achieve a 100% decoding accuracy without a TA. Generally, the longer the $l_m$, the higher the utilization. And when $l_m$ is greater than 32, the improvements in both speed and utilization become less significant. For the subsequent experiments, we set $l_m = 64$.

## 4.4 Comparison of Time and Speed Across Different Steganographic Methods

In this section, we compare the sampling time and generation speed of various steganographic methods based on the GPT-2 model under different sampling spaces (Top-$p$), as shown in Table 3. As the $p$-value increases, we observe that the Average Token Sampling Time (ATST) increases for all steganographic algorithms except for AC [59], Meteor (w/o sort) [28] and SparSamp. This increase in ATST can be attributed to the significant rise in the number of candidate tokens as the $p$-value grows. Methods that require sorting to achieve higher embedding rates, such as the sorted versions of Meteor and Discop [16], ADG, and iMEC [14], experience greater computational complexity. In contrast, SparSamp demonstrates the fastest sampling speed. When compared to normal generation (random sampling), it incurs only negligible additional sampling time, further confirming its $O(1)$ time complexity, as discussed in Section 3.3.

Additionally, we analyzed the ratio of sampling time to model inference time (SITR). In typical generation processes, sampling time constitutes only a tiny fraction of the total time, approximately 0.02 of the inference time. However, other steganographic methods exhibit a significant increase in sampling time as the candidate token space expands, sometimes reaching tens (ADG [56] and Discop [16]) or even hundreds (Meteor [28] and iMEC [14]) of times the inference time. In contrast, SparSamp maintains a sampling time nearly identical to normal sampling.

Table 2: The performance of the SparSamp with different $l_m$ using GPT-2 under $p = 1.00$.

| $l_m$ | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1023 | $\geq 1024$ |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Utilization ↑ | 27.5% | 44.7% | 64.5% | 78.8% | 87.3% | 97.4% | 98.0% | 98.5% | 98.7% | **99.5%** | / |
| Embedding Speed ↑ (bits/s) | 214.7 | 358.4 | 504.0 | 640.4 | 705.1 | **755.4** | 731.0 | 656.3 | 709.7 | 706.0 | / |
| Decoding Speed ↑ (bits/s) | 203.7 | 339.7 | 477.7 | 606.7 | 667.3 | **715.6** | 699.6 | 629.9 | 678.0 | 672.4 | / |
| Decoding Accuracy | 100% | 100% | 100% | 100% | 100% | 100% | 100% | 100% | 100% | 100% | 0% |

Table 3: Comparison of ATST, SITR, and generation speed for different steganographic methods

| GPT-2 | $p$ | Random Sampling | ADG [56] | AC [59] | Meteor [28] | | Discop [16] | | iMEC [14] | SparSamp |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | sort | w/o sort | sort | w/o sort | | |
| ATST ↓ (s/token) | 0.80 | 1.31E-04 | 1.13E-03 | 1.78E-03 | 2.83E-02 | 1.67E-03 | 6.66E-04 | 3.96E-04 | 3.64E-03 | **1.53E-04** |
| | 0.95 | 1.32E-04 | 8.91E-03 | 1.76E-03 | 2.86E-01 | 1.68E-03 | 5.71E-03 | 6.83E-04 | 1.46E-02 | **1.57E-04** |
| | 1.00 | 6.41E-04 | 6.67E-01 | 2.41E-03 | 5.00E+00 | 2.26E-03 | 2.78E-01 | 1.04E-02 | 2.00E+00 | **7.21E-04** |
| SITR ↓ | 0.80 | 0.02 | 0.15 | 0.23 | 3.71 | 0.22 | 0.09 | 0.05 | 0.48 | **0.02** |
| | 0.95 | 0.02 | 1.19 | 0.24 | 38.12 | 0.22 | 0.76 | 0.09 | 1.94 | **0.02** |
| | 1.00 | 0.09 | 97.41 | 0.35 | 730.57 | 0.33 | 40.59 | 1.52 | 292.23 | **0.11** |
| Generation Speed ↑ (tokens/s) | 0.80 | 128.9 | 118.3 | 109.7 | 27.7 | 110.9 | 123.5 | 125.0 | 88.0 | **125.8** |
| | 0.95 | 131.1 | 61.5 | 109.5 | 3.4 | 110.5 | 75.8 | 122.4 | 45.2 | **130.6** |
| | 1.00 | 133.6 | 1.5 | 109.9 | 0.2 | 111.7 | 3.5 | 52.8 | 0.5 | **132.3** |

Table 4: Comparison of embedding capacity and speed for different steganographic methods

| GPT-2 | $p$ | *Random (Entropy)* | ADG [56] | AC [59] | Meteor [28] | | Discop [16] | | iMEC [14] | SparSamp |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | sort | w/o sort | sort | w/o sort | | |
| Embedding Rate (bits/token) | 0.80 | 3.69 | 3.01 | 3.85 | 3.12 | 2.63 | 3.65 | 1.86 | 2.86 | 3.60 |
| | 0.95 | 5.22 | 4.16 | 5.31 | 4.45 | 3.71 | 5.04 | 2.09 | 3.62 | 5.16 |
| | 1.00 | 6.00 | 4.76 | 5.60 | 4.95 | 4.17 | 5.63 | 2.24 | 4.13 | 5.98 |
| Utilization ↑ | 0.80 | / | 73.9% | **99.3%** | 79.6% | 63.8% | 92.3% | 47.5% | 76.8% | 95.3% |
| | 0.95 | / | 79.1% | **99.4%** | 83.0% | 70.4% | 95.1% | 40.9% | 71.9% | 94.9% |
| | 1.00 | / | 90.0% | **99.6%** | 85.2% | 71.8% | 95.4% | 37.5% | 68.9% | 97.4% |
| Embedding Speed ↑ (bits/s) | 0.80 | / | 341.9 | 410.3 | 81.7 | 279.5 | 422.4 | 223.8 | 251.7 | **461.7** |
| | 0.95 | / | 258.3 | 567.2 | 15.1 | 410.0 | 370.7 | 259.5 | 163.6 | **628.2** |
| | 1.00 | / | 7.6 | 647.3 | 1.1 | 512.7 | 19.8 | 120.4 | 2.1 | **755.4** |
| Decoding Speed ↑ (bits/s) | 0.80 | / | 348.9 | 441.6 | 89.7 | 322.7 | **442.2** | 221.9 | 226.1 | 421.4 |
| | 0.95 | / | 247.7 | **639.9** | 14.7 | 475.9 | 386.2 | 246.2 | 123.0 | 560.4 |
| | 1.00 | / | 7.9 | **762.3** | 0.9 | 576.7 | 20.8 | 146.3 | 0.7 | 715.6 |

Table 5: Comparison of KLD for different steganographic methods using GPT-2 under various $p$-values

| GPT-2 | $p$ | ADG [56] | AC [59] | Meteor w/ sort [28] | SparSamp |
|---|---|---|---|---|---|
| Ave / Max KLD (bit/token) | 0.80 | 7.70E-03 / 5.58E-02 | 5.08E-02 / 9.29E+00 | 5.15E-02 / 6.68E+00 | **0 / 0** |
| | 0.95 | 1.08E-02 / 5.22E-02 | 2.93E-03 / 2.52E+00 | 2.85E-03 / 1.80E+00 | **0 / 0** |
| | 1.00 | 1.26E-02 / 5.38E-02 | 1.79E-04 / 2.21E-01 | 1.13E-06 / 1.62E-05 | **0 / 0** |

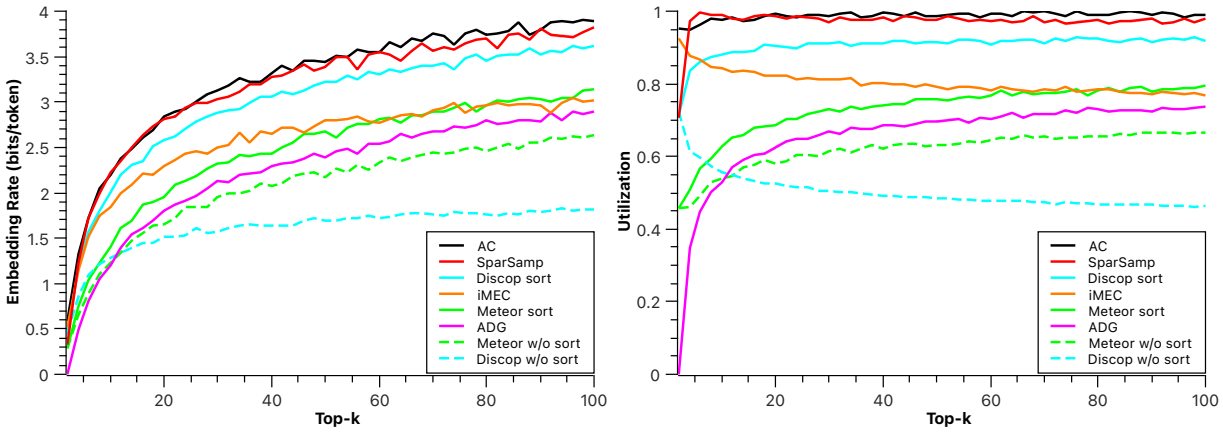



Figure 7: Comparison of embedding rate and utilization for various steganographic methods in low-entropy environments using GPT-2 with Top-$k$ sampling

## 4.5 Comparison of Embedding Capacity and the Speed of Embedding and Decoding Across Different Steganographic Methods

This section compares the embedding capacity and speed of various steganographic methods using the GPT-2 model across different sampling spaces, as shown in Table 4. As the $p$ increases, the embedding rate for all steganographic methods rises. This increase is attributed to the higher information entropy of the token probability distribution $P_i$ at each generation step with higher $p$, as the random sampling shows.

However, the embedding rate alone does not accurately reflect the embedding capacity, as the embedded messages are random and the sampled tokens differ, leading to varying entropy levels at each step. Therefore, we also take into account the entropy utilization rate. The AC method [59] shows the highest entropy utilization, which aligns with its performance in compression tasks. SparSamp generally has the second-highest utilization, following AC.

In terms of embedding speed, SparSamp significantly outperforms existing methods, achieving a embedding speed of 755.4 bits/s on GPT-2. This performance advantage stems from its $O(1)$ complexity, which eliminates the need for computationally intensive operations like probability reordering or interval scaling. Furthermore, SparSamp maintains a comparable decoding speed, ensuring rapid message extraction without error.

In particular, we compared the embedding rate and utilization in low-entropy environments. To create such an environment, we employed top-$k$ sampling with GPT-2, setting the $k$ value range from 2 to 100. In this setting, each sampling step had only $k$ candidate tokens, resulting in significantly lower entropy compared to top-$p$ sampling. As shown in Figure 7, both SparSamp and AC [59] demonstrated high embedding rates across different $k$ values. For $k$ values greater than 2, SparSamp's utilization approached 1. In contrast, the utilization of other methods (except for Discop w/o sort and iMEC), gradually increased with higher k values, reaching a maximum of only 0.9.

## 4.6 Comparison of Security

In this section, we compare the KLD induced by different steganographic methods based on GPT-2 under various $p$-values, as shown in Table 5. Notably, SparSamp maintains zero average and maximum KLD due to its preservation of distribution probabilities. In contrast, AC [59], Meteor [28], and ADG [54] introduce varying degrees of KLD. Particularly, the KLD of these three methods is significant, potentially allowing adversaries to gain a non-negligible advantage in detection.

To complement our theoretical security proof of SparSamp and ensure a comprehensive evaluation, we conducted empirical tests using various established steganalysis methods. These tests aimed to distinguish between cover text generated by random sampling and stego text produced by SparSamp. We generated 10,000 pairs of cover and stego texts under the truncation parameter $p = 0.95$. We employed three deep learning-based steganalysis methods that have demonstrated effectiveness in benchmark tests: FCN [55], R-BiLSTM-C [35], BiLSTM-Dense [52]. Table 6 presents the results of our analysis. Notably, the detection error rate $P_E$ for SparSamp approaches 50%. Since a bias of less than 0.5% can currently be considered negligible in steganalysis, our experimental results (<0.5%) can be considered indistinguishable. This indicates that the steganalysis methods perform no better than random guessing in identifying stego content generated by SparSamp, thereby empirically confirming its security.

Table 6: Steganalysis results for SparSamp

| Methods | FCN [55] | R-BiLSTM-C [35] | BiLSTM-Dense [52] |
|---|---|---|---|
| $P_E$ | 49.63% | 50.09% | 49.58% |

Table 7: Performance of SparSamp across different models

| Model | Embedding Speed (bits/s) | Embedding Rate (bits/token) | Utilization |
|---|---|---|---|
| Qwen-2.5 | 117.2 | 3.31 | 95.8% |
| Llama-3 | 90.1 | 2.66 | 96.6% |
| DDPM | 5046.8 | 2.33 | 98.5% |
| WaveRNN | 9223.4 | 4.83 | 96.5% |

## 4.7 Deployment on Different Generative Models

We deployed SparSamp on four additional models: the recently open-sourced LLM Qwen-2.5 [51] and Llama-3 [2], the image generation model DDPM [11], and the audio generation model WaveRNN [27]. The implementation was straightforward, requiring only the replacement of the sampling component in these models with SparSamp encoding.

Table 7 illustrates SparSamp's performance across these various models, with utilization consistently exceeding 95% for all of them. The embedding speed achieved with the WaveRNN model was particularly impressive, reaching 9,223 bits/s. This remarkable speed opens up the possibility for real-time, high-capacity covert communication. Based on the DDPM, the embedding rate we can achieve is 2.33 bits/token, which means we can embed a 0.29 size of the secret image in the stego image. Examples of stego texts and stego images generated with the LLM and DDPM can be found in Appendix A.

## 4.8 Evaluating SparSamp in combination with the Token Disambiguating Technique

In previous experiments, we demonstrated that with no TA, the decoding accuracy of SparSamp reached 100%, and its decoding speed was exceptionally fast. When there is TA, existing steganographic coding [14, 16, 28, 56, 59] all face the problem of being unable to decode accurately. However, it can be resolved by the token disambiguating technique without changing the probability distribution and sacrificing the security [5, 40].

We first use BackCheck [5] to deal with the TA. We embedded 64-bit messages at a time and generated 10,000 pairs of stego texts based on different LLMs and steganography methods, and we counted the frequency of TA. As quantified in Table 8, the TA occurrence rate per 64-bit embedding remains exceptionally low—ranging from 0.67% to 3.81% depending on the LLM architecture.

Table 8: Analyze the token ambiguity with BackCheck [5]

| Model | Frequency of TA | AC [59] / SparSamp | |
|---|---|---|---|
| | | Frequency of detecting TA | Average distance from TA |
| GPT-2 | 2.49% | 0% / **79.3%** | 5.5 / **3.6** |
| Qwen-2.5 | 0.67% | 0% / **73.2%** | 8.0 / **6.6** |
| Llama-3 | 3.81% | 0% / **65.4%** | 10.1 / **6.9** |

Table 9: Performance of SparSamp with SynPool [40]

| Model ($p = 0.8$) | GPT-2 | Qwen-2.5 | Llama-3 |
|---|---|---|---|
| Embedding Rate | 2.41 | 1.35 | 0.70 |
| Utilization | 72.8% | 65.4% | 70.2% |

When BackCheck is used without considering checkpoints, the AC-based method is unable to detect the presence of TA. In contrast, SparSamp can identify the presence of TA with a probability of 65% to 79%, i.e., $N_m = 0$ when extracted according to Algorithm 4. This ability allows for shorter checkpoints and enhances the embedding capacity according to [5]. In addition, we assess the distance from TA of a token encountered with $N_m = 0$ during SparSamp extraction. Using SparSamp reduces the number of queries for two tokens on average compared to the number of queries needed to locate the TA after the AC has extracted the message. Consequently, SparSamp requires only about half the length of the checkpoint to detect the location of the TA faster than AC, and it also reduces the impact on the embedding capacity. We verified that for every 60-bit embedding, only 4 bits of the checkpoint are needed to backtrack to the correct path thus enabling accurate extraction based on SaprSamp.

We also use SynPool [40] to deal with the TA. The results are shown in Table 9. SynPool is suitable for settings in LLM with a low $p$ value because a high $p$ value reduces utilization according to [40].

Finally, we would like to say that TA is not as bad as we think, and we counted that most TA is caused by some special symbols (e.g., ".....","")..."), rather than words. And these special symbols are relatively easy to find the correct parsing.

## 5 Conclusion

We propose SparSamp, a novel PSS that employs message-driven sampling for embedding messages, achieving unambiguous message embedding and extraction with high embedding rate. We demonstrate that SparSamp introduces minimal computational overhead, with an added complexity of only $O(1)$. Crucially, SparSamp preserves the original probability distribution, ensuring provable security. We demonstrate

the performance of the SparSamp through text, image and speech generation tasks, and the experimental results show that SparSamp offers high capacity, rapid decoding speed and the fastest embedding speed among comparable methods. In the future, we hope to further extend SparSamp to realize robust PSS and build public-key PSS.

## Acknowledgments

We thank the reviewers for their valuable comments. This work was supported in part by the Natural Science Foundation of China under Grant 62302146, 62472398, U2336206, and U2436601.

## 6 Ethics Considerations

We used open-source models and datasets for research transparency. SparSamp efficiently embeds messages while optimizing time and resources. Our ethical approach ensures secure communication with minimal computational impact. We aim to mitigate potential negative outcomes.

Steganography is a double-edged sword. It enables secure, covert communication. However, it also raises ethical concerns, as it can be misused for illegal activities or spreading misinformation. Steganography complicates law enforcement efforts by obscuring communication content and intent. While valuable for clandestine communication, it poses risks that require careful societal consideration.

## 7 Open Science

We hereby commit to full compliance with the open science policy. We acknowledge the importance of transparency and reproducibility in research. Our implementation exclusively utilizes open-source models (GPT-2, Qwen-2.5, Llama-3, DDPM, WaveRNN) and publicly available datasets (IMDB, FFHQ) to ensure reproducibility. Our research artifacts related to the SparSamp are openly available at https://doi.org/10.5281/zenodo.14736967. The artifact provides a Python implementation for encoding and decoding messages using the SparSamp method based on the generative model. The core functionalities are encapsulated in the `encode_spar` and `decode_spar` functions. We believe that sharing our research artifacts will not only enhance the validity of our work but also foster collaboration and innovation in the field of steganography.

## References

[1] https://llama.meta.com.

[2] AI@Meta. Llama 3 model card. 2024.

[3] R.J. Anderson and F.A.P. Petitcolas. On the limits of steganography. *IEEE Journal on Selected Areas in Communications*, 16(4):474–481, 1998.

[4] Sercan Ö. Arik, Mike Chrzanowski, Adam Coates, Gregory Frederick Diamos, Andrew Gibiansky, Yongguo Kang, Xian Li, John Miller, Andrew Ng, Jonathan Raiman, Shubho Sengupta, and Mohammad Shoeybi. Deep voice: Real-time neural text-to-speech. *ArXiv*, abs/1702.07825, 2017.

[5] Luke A. Bauer, James K. Howes, Sam A. Markelon, Vincent Bindschaedler, and Thomas Shrimpton. Leveraging generative models for covert messaging: Challenges and tradeoffs for "dead-drop" deployments. In *Proceedings of the Fourteenth ACM Conference on Data and Application Security and Privacy*, CODASPY '24, page 67–78, New York, NY, USA, 2024. Association for Computing Machinery.

[6] Mehdi Boroumand, Mo Chen, and Jessica Fridrich. Deep residual network for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 14(5):1181–1193, 2019.

[7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. J. Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *ArXiv*, abs/2005.14165, 2020.

[8] Christian Cachin. An information-theoretic model for steganography. In *Information and Computation*, 1998.

[9] Kejiang Chen, Hang Zhou, Dongdong Hou, Hanqing Zhao, Weiming Zhang, and Nenghai Yu. Provably secure steganography on generative media. *CoRR*, abs/1811.03732, 2018.

[10] Kejiang Chen, Hang Zhou, Hanqing Zhao, Dongdong Chen, Weiming Zhang, and Nenghai Yu. Distribution-preserving steganography based on text-to-speech generative models. *IEEE Transactions on Dependable and Secure Computing*, 2021.

[11] Jooyoung Choi, Sungwon Kim, Yonghyun Jeong, Youngjune Gwon, and Sungroh Yoon. Ilvr: Conditioning method for denoising diffusion probabilistic models. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 14347–14356, 2021.

[12] David Cole. We kill people based on metadata. https://www.nybooks.com/online/2014/05/10/we-kill-people-based-metadata/, May 2014.

[13] United States Congress. Kids online safety act. https://www.congress.gov/bill/118th-congress/house-bill/7891, April 2024.

[14] Christian Schroeder de Witt, Samuel Sokota, J Zico Kolter, Jakob Nicolaus Foerster, and Martin Strohmeier. Perfectly secure steganography using minimum entropy coupling. In *The Eleventh International Conference on Learning Representations*, 2023.

[15] Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis. *ArXiv*, abs/2105.05233, 2021.

[16] Jinyang Ding, Kejiang Chen, Yaofei Wang, Na Zhao, Weiming Zhang, and Neng H. Yu. Discop: Provably secure steganography in practice based on "distribution copies". *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2238–2255, 2023.

[17] Tomáš Filler, Jan Judas, and Jessica Fridrich. Minimizing additive distortion in steganography using syndrome-trellis codes. *IEEE Transactions on Information Forensics and Security*, 6(3):920–935, 2011.

[18] Jessica Fridrich. Minimizing the embedding impact in steganography. In *Proceedings of the 8th Workshop on Multimedia and Security*, MM&Sec '06, page 2–10, New York, NY, USA, 2006. Association for Computing Machinery.

[19] Philip Gage. A new algorithm for data compression. *C Users J.*, 12(2):23–38, February 1994.

[20] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014.

[21] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS'20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[22] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.

[23] Ari Holtzman, Jan Buys, Maxwell Forbes, Antoine Bosselut, David Golub, and Yejin Choi. Learning to write with cooperative discriminators. In Iryna Gurevych and Yusuke Miyao, editors, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1638–1649, Melbourne, Australia, July 2018. Association for Computational Linguistics.

[24] Nicholas Hopper, Luis von Ahn, and John Langford. Provably secure steganography. *IEEE Transactions on Computers*, 58(5):662–676, 2009.

[25] Nicholas J Hopper. *Toward a theory of Steganography*. Carnegie Mellon University, 2004.

[26] Nicholas J. Hopper, John Langford, and Luis von Ahn. Provably secure steganography. In Moti Yung, editor, *Advances in Cryptology — CRYPTO 2002*, pages 77–92, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.

[27] Nal Kalchbrenner, Erich Elsen, Karen Simonyan, Seb Noury, Norman Casagrande, Edward Lockhart, Florian Stimberg, Aaron van den Oord, Sander Dieleman, and Koray Kavukcuoglu. Efficient neural audio synthesis. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 2410–2419. PMLR, 10–15 Jul 2018.

[28] Gabriel Kaptchuk, Tushar M. Jois, Matthew Green, and Aviel D. Rubin. Meteor: Cryptographically secure steganography for realistic distributions. *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021.

[29] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4396–4405, 2018.

[30] Andrew D Ker. Improved detection of lsb steganography in grayscale images. In *International workshop on information hiding*, pages 97–115. Springer, 2004.

[31] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[32] Tri Van Le. Efficient provably secure public key steganography. Cryptology ePrint Archive, Paper 2003/156, 2003. https://eprint.iacr.org/2003/156.

[33] Maciej Liśkiewicz, Rüdiger Reischuk, and Ulrich Wölfel. Grey-box steganography. In Mitsunori Ogihara and Jun Tarui, editors, *Theory and Applications of*

*Models of Computation*, pages 390–402, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[34] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In Dekang Lin, Yuji Matsumoto, and Rada Mihalcea, editors, *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, Portland, Oregon, USA, June 2011. Association for Computational Linguistics.

[35] Yan Niu, Juan Wen, Ping Zhong, and Yiming Xue. A hybrid r-bilstm-c neural network based text steganalysis. *IEEE Signal Processing Letters*, 26:1907–1911, 2019.

[36] Jumon Nozaki and Yugo Murawaki. Addressing segmentation ambiguity in neural linguistic steganography. In Yulan He, Heng Ji, Sujian Li, Yang Liu, and Chua-Hui Chang, editors, *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 109–116, Online only, November 2022. Association for Computational Linguistics.

[37] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke E. Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Francis Christiano, Jan Leike, and Ryan J. Lowe. Training language models to follow instructions with human feedback. *ArXiv*, abs/2203.02155, 2022.

[38] United Kingdom Parliament. Online safety act 2023. https://bills.parliament.uk/bills/3137, 2023.

[39] Yinyin Peng, Donghui Hu, Yaofei Wang, Kejiang Chen, Gang Pei, and Weiming Zhang. Stegaddpm: Generative image steganography based on denoising diffusion probabilistic model. In *Proceedings of the 31st ACM International Conference on Multimedia*, MM '23, page 7143–7151, New York, NY, USA, 2023. Association for Computing Machinery.

[40] Yuang Qi, Kejiang Chen, Kai Zeng, Weiming Zhang, and Nenghai Yu. Provably secure disambiguating neural linguistic steganography. *IEEE Transactions on Dependable and Secure Computing*, pages 1–14, 2024.

[41] Alec Radford and Karthik Narasimhan. Improving language understanding by generative pre-training. 2018.

[42] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[43] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8821–8831. PMLR, 18–24 Jul 2021.

[44] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. *ArXiv*, abs/2102.12092, 2021.

[45] Gustavus J. Simmons. The prisoners' problem and the subliminal channel. In *Annual International Cryptology Conference*, 1983.

[46] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew W. Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *ArXiv*, abs/1609.03499, 2016.

[47] Tri Van Le and Kaoru Kurosawa. Bandwidth optimal steganography secure against adaptive chosen stegotext attacks. In Jan L. Camenisch, Christian S. Collberg, Neil F. Johnson, and Phil Sallee, editors, *Information Hiding*, pages 297–313, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.

[48] Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Neural Information Processing Systems*, 2017.

[49] Yuxuan Wang, R.J. Skerry-Ryan, Daisy Stanton, Yonghui Wu, Ron J. Weiss, Navdeep Jaitly, Zongheng Yang, Ying Xiao, Zhifeng Chen, Samy Bengio, Quoc Le, Yannis Agiomyrgiannakis, Rob Clark, and Rif A. Saurous. Tacotron: Towards End-to-End Speech Synthesis. In *Proc. Interspeech 2017*, pages 4006–4010, 2017.

[50] Ruiyi Yan, Yating Yang, and Tian Song. A secure and disambiguating approach for generative linguistic steganography. *IEEE Signal Processing Letters*, 30:1047–1051, 2023.

[51] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru

Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[52] Hao Yang, YongJian Bao, Zhongliang Yang, Sheng Liu, Yongfeng Huang, and Saimei Jiao. Linguistic steganalysis via densely connected lstm with feature pyramid. *Proceedings of the 2020 ACM Workshop on Information Hiding and Multimedia Security*, 2020.

[53] Kuan Yang, Kejiang Chen, Weiming Zhang, and Nenghai Yu. Provably secure generative steganography based on autoregressive model. In *International Workshop on Digital Watermarking*, 2018.

[54] Kuan Yang, Kejiang Chen, Weiming Zhang, and Nenghai Yu. Provably Secure Generative Steganography Based on Autoregressive Model. In Chang D. Yoo, Yun-Qing Shi, Hyoung Joong Kim, Alessandro Piva, and Gwangsu Kim, editors, *Digital Forensics and Watermarking*, pages 55–68, Cham, 2019. Springer International Publishing.

[55] Zhongliang Yang, Yongfeng Huang, and Yujin Zhang. A fast and efficient text steganalysis method. *IEEE Signal Processing Letters*, 26:627–631, 2019.

[56] Siyu Zhang, Zhongliang Yang, Jinshuai Yang, and Yongfeng Huang. Provably secure generative linguistic steganography. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3046–3055, Online, August 2021. Association for Computational Linguistics.

[57] Weiming Zhang and Xin Wang. Generalization of the zzw embedding construction for steganography. *Trans. Info. For. Sec.*, 4(3):564–569, sep 2009.

[58] Xinpeng Zhang and Shuozhong Wang. Efficient steganographic embedding by exploiting modification direction. *IEEE Communications Letters*, 10(11):781–783, 2006.

[59] Zachary M. Ziegler, Yuntian Deng, and Alexander M. Rush. Neural linguistic steganography. *CoRR*, abs/1909.01496, 2019.

## A Examples of Output of SparSamp

This appendix contains stego text and image outputs generated by SparSamp using several different model types.

Table 10 gives examples of employing different LLMs to perform secret message encoding and stego text generation. We set the truncation $p = 1$. We first use the model itself to encode the secret message into a bit string based on AC as in [59]. Then, we embed the bit string by generating the stego text using the same LLM with SparSamp. We can see that LLMs such as Qwen-2.5 and Llama-3 generate a longer stego text but more realistic content.

Figure 8 gives examples of employing DDPM [11] to perform image generation. The DDPM is pre-trained on the FFHQ dataset. SparSamp integration in the final sampling step of DDPM according to [39]. The size of the generated image is $256 \times 256$. An average of 457,000 bits can be embedded in each image. The embedding rate is 2.33 bits/token. This means that we can losslessly embed an image of the same type with 0.29 times the number of pixels of a stego image.

Table 10: Stego text generated by different LLM with SparSamp

**Secret message:**
In a strategic military operation, soldiers were tasked with disrupting enemy supply lines by breaking a critical road with C4 explosives. Under cover of darkness, the soldiers approached the road, carefully placing the explosives at key points. With precision timing, they detonated the C4, creating massive craters that rendered the road impassable. The explosion's force was so great that it could be heard for miles around. The operation was a success, cutting off enemy forces from vital resources and slowing their advance. The soldiers withdrew safely, leaving behind a road turned into a wasteland of debris and destruction.

**Length of bit string after encoding the secret message using GPT-2 with AC: 533**
**Prompt: Once upon a time,**
**Stego text generated by GPT-2 with SparSamp:**
evidence here bearing against Los Angeles had accumulated, especially in evidence van Creussel went to the Lens in Los Angeles for evidence covering his trial that fell onto the paper. We were looking for witnesses, not documents. This was the obvious case, and he was acquitted. Given the having uncovered all this evidence, and also the claim– by Los Angeles law firm colleagues in 1980 that he was ordered to pay bribe when bribe gained confidence–that a certified deposition must form the basis for conviction

**Length of bit string after encoding the secret message using Qwen-2.5 with AC: 360**
**Prompt: Give me a short introduction to large language model.**
**Stego text generated by Qwen-2.5 with SparSamp:**
Large language models (LLMs) are a type of artificial intelligence designed to process and generate human-like language. These models operate on vast amounts of text data, often exceeding the scope of the training dataset, achieving impressive natural language understanding and generation performance. Key characteristics of these models include:
1. **Capacity for Knowledge Expansion**: LLMs continuously expand their domains of understanding, complemented by continual pretraining. 2. **Enhanced Resource Use**: Unlike simpler models, LLMs can utilize larger corpora and more compute resources to perform well across various tasks. 3. **Multi-Aspect Handling**: They can handle syntax, semantics, and pragmatic components of language comprehensively. 4. **Task Adaptability**: Through fine-tuning, LLMs can be effectively retrained for specific applications such as language translation, summarization, question-answering, and creative writing.
These models are foundational for achieving advanced functionalities in natural language applications.

**Length of bit string after encoding the secret message using Llama-3 with AC: 336**
**Prompt: Why do dogs love to stick out their tongues in summer?**
**Stego text generated by Llama-3 with SparSamp:**
Here are the possible reasons: 1. Cooling down: Dogs don't have sweat glands all over their body like humans do, so they need to find other ways to cool down. Licking their body, including their tongue, helps to regulate their body temperature. As they exhale, their tongue picks up heat and dries out, making them feel cooler. 2. Hydration: Tongue-sticking and panting are two ways dogs regulate their hydration levels. By sticking their tongue out, they help to dissipate excess heat and humidity. This also helps them to rehydrate, as they lick their tongue and reabsorb moisture. 3. Social bonding: Sticking out their tongue, also known as ""flehmen response,"" is a natural behavior that can be a sign of bonding and affection in dogs. They may use it to communicate with their human family members or other pets. 4. Relaxation: Dogs often stick out their tongues when they're feeling relaxed and at ease. This could be due to the sensation of the tongue touching the air or the release of tension in their mouth. 5. Instinctual behavior: In the wild, dogs used to stick out their tongues to inspect their surroundings, detect prey, and gather information. This behavior has been passed down through generations, and even domesticated dogs exhibit it as a natural behavior. 6. Sensory experience: Sticking out their tongue can be a sensory experience for dogs. The sensation of the tongue touching air, the movement of their tongue, and the perception of their own breath may provide a calming or enjoyable sensation.

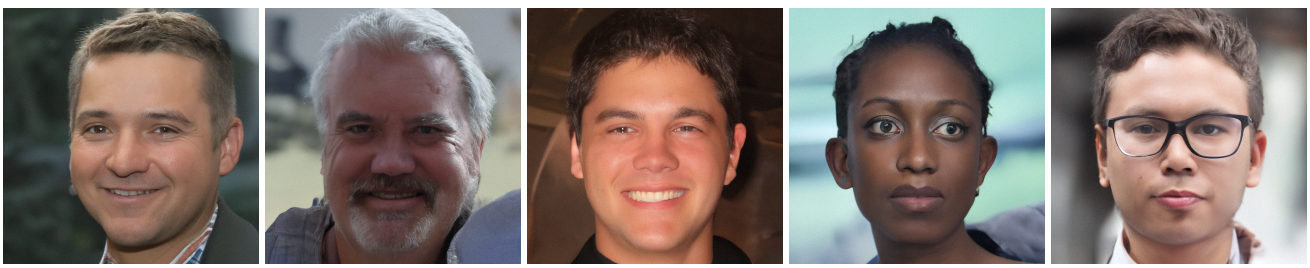


Figure 8: Stego image generated by DDPM with SparSamp